

42417 — Complete Study Notebook

Simulation in Operations Management — Midterm Theory

For the written midterm, Monday 15 June 2026, 10:00–11:00

This notebook explains every theoretical point in the conceptual curriculum in depth. Read it alongside the one-page cheat sheet: the cheat sheet is for the exam room, this notebook is for understanding *why* each point is true so the multiple-choice questions feel obvious.

1. What simulation is, and why we use it

Simulation is the mimicking of a real-world system or process over time. Instead of solving the system with closed-form mathematics, you build a model that imitates how the system actually behaves, and then you “run” that model to watch what happens. Rossetti frames it as the next best thing to observing the real system itself. A good simulation model represents the system so faithfully that its behaviour and qualities match those of the real thing.

Why not just use equations or a spreadsheet? Most real operations — a bank office, an airport apron, a production line — combine three features at once: *queueing*, *shared resources*, and *randomness*. When these interact, there is usually no analytical formula that gives the answer, and even if a formula existed for one configuration, a small process change would break it. Simulation handles this messiness directly.

Why simulate (the positive reasons):

- It lets you study and experiment with the *internal interactions* of a complex system that could not be studied any other way.
- You can simulate informational, organisational and environmental *changes* and observe their effects before committing to them in reality.
- The knowledge gained while designing the model can itself be used to improve the real system.
- The model lets you *trace the variables of importance* and see how they interact.
- It is a pedagogical device — it can reinforce and even verify analytical solution methods.

Advantages: flexibility to model things as they really are (even messy ones); it allows uncertainty and non-stationary (time-varying) behaviour; modern software makes it accessible; animation aids demonstration, verification and debugging; results are far more convincing to stakeholders than a spreadsheet.

Disadvantages: you do *not* get exact answers — only approximations and estimates, because every run is one random sample; and it requires careful statistical design and analysis of the experiments. This single fact — “estimates, not exact answers” — is the reason the entire discipline of input modelling, verification/validation and output analysis exists.

Physical vs. virtual simulation. Physical simulations are things like board games, military exercises and role-play (the “beer game”). Virtual simulations are computer models for complex analysis — which is what this course is about.

2. Classifying systems

You classify a system *before* modelling it, because the classification dictates which modelling approach and which analysis methods are appropriate. The taxonomy is a tree with three binary splits.

Top split — Stochastic vs. Deterministic. A **deterministic** model is one whose behaviour can be predicted precisely: given the inputs, the output is fixed and repeatable. A **stochastic**

model is one whose behaviour cannot be fully predicted because at least part of it follows probability distributions — run it twice with different random seeds and you get different results.

Middle split — Dynamic vs. Static. **Dynamic** means the state variables change *over time*. **Static** means you take a snapshot at a single instant; the classic static example is a *Monte Carlo* simulation, which samples a fixed random situation repeatedly without time playing a role.

Bottom split — Continuous vs. Discrete. In a **continuous** system the state variables change smoothly, passing through an infinite number of intermediate states (no abrupt jumps). In a **discrete-event** system the state variables change *only* at distinct points in time when an *event* occurs; between events, nothing changes.

The systems this course models — and what AnyLogic's Process Modelling library is built for — live in the **Stochastic → Dynamic → Discrete** branch. This is **Discrete-Event Simulation (DES)**. If an exam question describes a bank or airport with random arrivals and asks you to classify it, this is the answer.

3. The vocabulary of discrete-event simulation

Rossetti's core concepts give you the language to describe any DES model. Learn these precisely — the multiple-choice exam loves definition-matching.

- **Entities** — the objects of interest whose activities are being modelled. They typically “flow” through the system. Examples: the customers in the bank, the aircraft at the airport.
- **Resources** — operating objects that *constrain* the flow of entities. A resource is requested by an entity, *acquired* (seized), held for the duration of an activity, and then *freed* (released). Examples: the tellers in the bank, the gates at the airport.
- **Process** — what happens when an entity requires a resource to complete an activity. The bank's service process is: customer seizes a teller, is served for some service time, releases the teller.
- **Event** — an instantaneous occurrence that changes the system state (an arrival; a service completion).
- **State variable** — describes the system at a given moment (number in queue; number of busy tellers).
- **Queue** — holds entities that are waiting because the resource they need is currently busy.
- **Attribute** — a property carried by an individual entity (e.g. its arrival time, which you store yourself so you can later compute time-in-system).
- **Simulation clock** — advances from one event time to the next; time “jumps” rather than ticking continuously.

Mapping to AnyLogic blocks: *Source* creates entities → *Queue* holds waiting entities → *Service* (or the explicit *Seize-Delay-Release* combination) models the resource-using activity → *ResourcePool* defines the resource units → *SelectOutput* routes entities down different branches → *Sink* disposes of them. The exam may ask which block corresponds to which Rossetti concept.

4. DEGREE — the problem-solving frame

Before any simulation-specific steps, Rossetti places the whole effort inside a general problem-solving process whose acronym is **DEGREE**. It is a mindset for solving *any* operational problem; the six-phase methodology (next section) is the concrete simulation workflow that lives *inside* the DEGREE frame. Know what each letter means and the reasoning behind it.

- **D — Define the problem.** Take a step back and make sure you are solving the *right* problem, not merely a symptom. In the bank tutorial this means asking: why does a simulation make sense here? What are the bottlenecks? What are the KPIs and how do they perform? Defining the problem badly dooms everything downstream.
- **E — Establish measures of performance for evaluation.** Decide *how* you will judge success. The metrics must be coherent with the problem you just defined — if the problem is long waits, throughput alone is the wrong measure.
- **G — Generate alternative solutions.** Produce several candidate designs rather than fixating on the first idea. You cannot choose the best option if you only ever consider one.
- **R — Rank alternative solutions.** Order the candidates against the performance measures from step E, so the comparison is structured and defensible.
- **E — Evaluate and iterate during the process.** Simulation is iterative: you evaluate the process and loop back, refining until the model reaches the fidelity (level of realism) the decision requires.
- **E — Execute and evaluate the solution.** Implement the chosen solution in the real organisation, then check that the promised benefits actually materialised. The job is not done when the model runs — it is done when reality improves.

Tie-in to the methodology: the first “D” of DEGREE corresponds to Phase 1 of the General Simulation Methodology. DEGREE = the philosophy; the six phases = the procedure.

5. The General Simulation Methodology — six phases in depth

Rossetti structures a simulation study as an *iterative* six-phase process. You are explicitly required to know *all six phases*, even though the course concentrates its hands-on work on Phases 1–5. The methodology is a planning tool: it tells you, step by step, what to do and in what order, and crucially where to loop back when something fails.

Phase 1 — Problem definition, objectives & planning, conceptual model

This is the foundation, and the course repeatedly insists you do it thoroughly *before* touching AnyLogic. It contains three blocks:

- *Problem definition* — define the system and the problem, decide the desired outputs, and identify the types of scenarios you will want to test.
- *Setting of objectives & project planning* — this is where *performance management* happens: you formulate the KPIs using the S.M.A.R.T. technique (Section 9) and plan the project.
- *Creation of the conceptual model* — you draw context diagrams, flow charts and activity diagrams (Section 7) that capture the system before any code exists.

Concretely, by the end of Phase 1 you should have: (1) defined the problem, (2) defined the system, (3) established performance metrics, (4) built conceptual models (flow chart + activity diagram), and (5) documented your modelling assumptions.

Phase 2 — Data collection, model building, coding, verification & validation

This is the build phase, and it contains a *loop*. You collect data and build the model *in parallel* (you don’t have to wait for perfect data to start building), and you do the coding — also called *model translation*. Then you hit two sequential decision gates:

- **Verified?** — is the model coded correctly? If *No*, go back and fix the code.

- **Validated?** — does the model represent reality? If *No*, go back further and fix the model structure, the data, or the assumptions.

You only leave Phase 2 once *both* gates pass. This is where the verification/validation distinction (Section 6) lives operationally.

Phase 3 — Experimental design + production runs and analysis

With a trusted model, you design the experiment (which configurations, how many replications — Sections 14 and 18), run the production runs, and analyse the output (Sections 13–15).

Phase 4 — Decision gate: “More runs?”

After analysing, you ask whether you need to explore further configurations or scenarios. If *Yes*, you loop back to Phase 3. If *No*, you proceed. This loop is what makes the methodology genuinely iterative rather than linear.

Phase 5 — Document programs and report results

You document the model and report the findings — tables, figures, confidence intervals, recommendations. A result that is not communicated clearly is worthless.

Phase 6 — Implementation

You put the chosen solution into practice in the real organisation. This is outside the course’s hands-on focus (which is why the emphasis is Phases 1–5), but it is still part of the methodology you must be able to name and describe.

Memory aid: (1) Define + conceptualise → (2) Build + verify + validate → (3) Experiment + analyse → (4) More runs? → (5) Document + report → (6) Implement. The two “loop-back” points are inside Phase 2 (verify/validate) and at Phase 4 (more runs).

6. Verification vs. Validation vs. Calibration

This is the single most exam-likely conceptual point because the words are so easy to confuse. Get it reflexive.

- **Verification** answers: “*Did I build the model right?*” It is an *internal* check that the model as implemented matches the conceptual design — that the code is free of bugs and behaves the way you intended. You compare the model to its *specification*.
- **Validation** answers: “*Did I build the right model?*” It is an *external* check that the model’s behaviour corresponds to the behaviour of the *real system*. You compare the model to *reality*.
- **Calibration** is the act of *tuning the model’s parameters* so its output matches observed reality. It is the dial-turning process that moves you toward a valid model.

One-line memory hook: **Verification = model vs. specification** (built it right); **Validation = model vs. reality** (built the right thing); **Calibration = the tuning that gets you there**.

Supporting concept — assumptions. Rossetti distinguishes two kinds of modelling assumption that you document and check:

- **Structural assumptions** — is the system operating the way I’ve represented it? Are my simplifications and abstractions of reality well chosen? These are verified through observation and discussion with experts.
- **Data assumptions** — is the data reliable, and is the statistical analysis correct? Two specific

requirements: if you *combine two datasets* collected at different times, you must test them for **homogeneity** (that they come from the same parent population); and if you *estimate probability distributions*, the data must form an **i.i.d.** sample (independent and identically distributed) — this is what the chi-square goodness-of-fit test and unbiased parameter estimation require.

7. Conceptual modelling diagrams

The conceptual model (built in Phase 1) is communicated through two related diagrams.

Context diagram / flow chart. A high-level picture of the system: its boundary, the inputs that come from the environment, the outputs it produces, and the broad flow of entities through it. It answers “what is in the system and what crosses its boundary?”

Activity diagram. A translation of the flow chart into a depiction of how *entities move through the system*. It makes explicit, for each step: where an entity *enters a queue*; where it *seizes* a resource to *start* an activity (only possible if the resource can be acquired); the activity’s *duration* (often a distribution, e.g. “service delay $\sim$ expo(3) minutes”); and where it *releases* the resource (*ends* the activity) and is finally *disposed*.

The activity diagram is the bridge between the conceptual picture and the AnyLogic implementation: every element (create / enter queue / seize / delay / release / dispose) corresponds directly to a process-modelling block. Drawing it well is what makes the coding straightforward.

8. Bottlenecks and performance management

Bottleneck. A **bottleneck** is the stage in a process that *limits the throughput of the whole system* — the slowest step, whose capacity caps everything else. The governing principle is that a chain of sequential stages can only produce as fast as its slowest member. Identifying the bottleneck is frequently the entire point of an operations simulation, because relieving the bottleneck is where investment actually pays off; upgrading a non-bottleneck stage achieves nothing for overall throughput. (Worked intuition: if one branch of a line tops out at 1,000 units/hour and another at 1,200, the 1,000 branch governs total output, so the machine responsible for that cap is the correct upgrade target.)

Performance management. This is the practice of *measuring* how well the system meets its objectives, operationalised through KPIs. The typical performance measures in these systems are:

- **Throughput** — units produced (or served) per unit time.
- **Utilisation** — the fraction of time a resource is busy.
- **Queue / waiting time** — how long entities wait for a resource.
- **Lead / turnaround time** — the total time an entity spends in the system from entry to exit.

9. S.M.A.R.T. KPIs (Doran, 1981) — in depth

A KPI (Key Performance Indicator) is only useful if it is well constructed. Doran’s **S.M.A.R.T.** technique gives five criteria a good KPI should satisfy. The exam may show you a candidate KPI and ask which criterion it fails, so understand each one, not just the word.

- **S — Specific.** To what degree is the goal precisely defined? A specific KPI pinpoints exactly what is being measured, leaving no ambiguity. “Reduce average customer waiting time at the teller” is specific; “improve the bank” is not.
- **M — Measurable.** Can a concrete metric be applied — of quality, quantity, cost, or time? If you cannot attach a number to it, you cannot track progress or know when it is met.

- **A — Attainable.** Is the goal within the capabilities of the person or department responsible? It should be *actionable* and *achievable* — a KPI nobody can influence is pointless.
- **R — Relevant.** Is it appropriate to the person's role and *realistic* for the problem at hand? The KPI must actually bear on the objective you defined in Phase 1.
- **T — Time-bound.** Is there an agreed time scale within which it should be achieved? Without a deadline or measurement window, a target cannot be evaluated.

SMART example: “Average aircraft turnaround time should be below 75 minutes, measured over a full operating day.” — Specific (turnaround time), Measurable (minutes), Attainable (within ground-handling control), Relevant (ties to the gate-bottleneck problem), Time-bound (per operating day). **Not SMART:** “Make the airport more efficient.” Note: SMART originated as a management/marketing tool (Doran 1981) and has been revised many times; the course uses it for KPI development.

10. Input modelling (Rossetti 6.1–6.4)

Once the model structure exists, you must feed it realistic inputs — this is **input modelling**, explicitly flagged as exam material.

Two kinds of input.

- **Controlled (known) inputs** — determined by the context of the model; *you* set them. Examples: number of employees, number of gates at an airport. These are your decision variables / factors.
- **Uncertain inputs** — unknown characteristics that must be represented by a distribution: either an *empirical* distribution drawn from observed data, or an *assumed theoretical* distribution whose parameters you estimate. Examples: customer arrival times, service times.

The typical input-modelling process (8 steps, Rossetti):

1. Document the process being modelled.
2. Develop a plan for collecting the data.
3. Collect the data (obtain any necessary approvals for processing).
4. Perform graphical and statistical analysis of the data.
5. Hypothesise possible distributions.
6. Decide / estimate the distribution parameters.
7. Check the goodness of fit of the hypothesised distributions.
8. Check the sensitivity of the simulation *outputs* to these inputs.

Empirical vs. theoretical distributions. A **theoretical** distribution is generally preferable *when one fits well*, because it is smooth and can generate values across its whole supported range. An **empirical** distribution that samples directly from your observed data can *always* be used, but it is limited: it can only reproduce values within the range you actually observed, so it cannot generate the rare extreme you never happened to record — it must be used carefully.

Choosing a theoretical distribution — consider its range:

- *Infinite both directions:* Normal.
- *Positive only:* Exponential, Gamma, Erlang, Weibull, Lognormal.
- *Bounded:* Uniform, Beta, Triangular.

You also weigh how easily a distribution's parameters let you manipulate the mean and variance, and you watch for outliers and multimodal data (which may justify splitting the dataset).

Two practical cautions: (1) Never fit a *continuous* distribution to *discrete* data. (2) If you have *no* data at all, you can still build a distribution from expert opinion — e.g. a Uniform distribution needs only a plausible minimum and maximum. And: don't let the fitting software replace your own judgement; test sensitivity first, collect better data only if the outputs are sensitive to that input.

11. Goodness of fit

How do you know a hypothesised distribution actually fits your data? Two statistical tests.

The tests. The **Chi-square** (χ^2) test (also called Pearson's chi-squared) and the **Kolmogorov–Smirnov (K–S)** test both check whether your dataset could plausibly have come from the candidate distribution. Each returns a *test statistic* and a **p-value**, measuring how far the dataset is from what the candidate distribution would produce.

Interpreting the p-value. The p-value is the likelihood of observing a result *at least as extreme* as your data, *assuming the null hypothesis is true*. If the p-value falls below the chosen **significance level** (typically 1% or 5%), you *reject* the null hypothesis. The significance level is the threshold of risk you accept.

The crucial subtlety (a classic exam trap): these tests can tell you to *reject* a poor fit, but they *cannot* be used to *rank or compare* how good competing fits are. A low p-value rejects a model — but neither the p-value nor the test statistic is a “quality score” for choosing between two models that both pass.

Number of bins (χ^2 specifically). Too *few* bins over-aggregates the data, lowers the number of comparison points, and reduces fitting quality. Too *many* bins shrinks the bars and their significance, increasing the error. The bin count must suit the number and variety of data points.

The Data Distribution Fitter tool. The course's tool tests nine distributions (Uniform, Normal, Exponential, Erlang, Weibull, Gamma, Lognormal, Triangular, Beta). For each it finds the best parameters (“fitting the distribution”), then *selects* the best model by **minimising the mean squared error** — the average gap between the fitted curve and the histogram bars. Note this selection criterion is *separate* from the hypothesis tests above.

12. Randomness in simulation (Rossetti 2.1–2.3)

Everything stochastic rests on the generation of randomness. The simulation *assumes* the randomness affecting the system can be described by probability distributions, and it produces random variate values by transforming streams of *pseudo-random numbers*. The practical consequences you must grasp:

- Every single run is just *one realisation* of a random process — which is exactly why you cannot trust one run (see Section 13).
- Controlling the random **seed** (or stream) lets you *reproduce* a run exactly — essential for debugging and fair comparison.
- Using a **common random number (CRN)** stream across two competing configurations makes the comparison *fairer*, because both configurations experience the “same” randomness, so observed differences are due to the design, not to luck.

13. Output analysis — why replications matter

Because the model is stochastic, a single run's output is just one noisy sample and must not be trusted alone. The course's motivating example is a modified Job Shop whose KPI varied between **129 and 151 minutes** across 100 runs — a $\pm 8\%$ swing around the mean of 140. Basing a decision on one run risks being 8% wrong, which would likely lead to wrong decisions.

The remedy: replications. Run the model many times with *different random seeds* but *identical input parameters*, and report the **across-replication mean** rather than any single run. The across-replication mean for replication i is the running average $\bar{X}_i = \frac{1}{i} \sum_{j=1}^i X_j$.

The steady-state plot. A plot of this running average versus the number of replications. The first value is the value itself, the second is the average of the first two, and so on. When the curve *flattens out* and stops changing much, you have enough replications (in the Job Shop example, around 70).

14. Determining the required number of replications

The reliability of the across-replication mean is quantified by a **confidence interval**, whose $\pm$ term is the **half-width**. The logic rests on the **Central Limit Theorem**: as the number of replications grows, the distribution of the sample mean converges to a normal distribution, which lets you state a range that contains the true KPI value with probability $(1 - \alpha)$. The course gives three ways to find the required number of replications n .

(a) Rossetti's half-width method (uses the t-distribution)

Valid because the across-replication KPI distribution is approximately normal. Procedure: run a pilot of n_0 replications ($n_0 = 15$ is the standard choice), compute the mean and the sample standard deviation s , choose your confidence level (hence α), and compute the half-width *experienced* in the pilot:

$$h_0 = t_{\alpha/2, n_0-1} \frac{s}{\sqrt{n_0}}$$

Then, given the half-width h you actually *desire*, the required number of replications is:

$$n \approx n_0 \left(\frac{h_0}{h} \right)^2$$

The unit of the half-width always follows the unit of the KPI.

Worked airport example: pilot of $n_0 = 15$ gave mean turnaround = 75.28 min, $s = 5.72$, experienced $h_0 = 3.17$ min, desired $h = 1.50$ min $\Rightarrow n \approx 67$.

(b) Ross's iterative method (uses the standard normal)

Fix an accepted standard-deviation threshold h and, using the standard normal distribution as the multiplier, check the inequality:

$$\Phi^{-1}\left(1 - \frac{\alpha}{2}\right) \frac{s}{\sqrt{n_0}} < h$$

If satisfied, n_0 is enough; if not, increase n_0 and repeat. In practice you run a large pilot (e.g. 200 reps in Excel) and iterate the calculation without re-running.

(c) Single combined formula

Derived from both Rossetti and Ross. Because the t-distribution converges to the standard normal as n grows, you can use:

$$n \geq \left(\frac{\Phi^{-1}\left(1 - \frac{\alpha}{2}\right) s}{h} \right)^2$$

Here s comes from a small pilot ($n_0 = 5, 10$ or 15). *Caveat:* only use this when n is large (e.g. $n \geq 50$), because the t-to-normal convergence requires a large sample.

Multiple KPIs: each method sizes n on *one* KPI. So either (i) compute n for every KPI and take the *largest*, ensuring confidence across all; or (ii) size n on the single most important KPI, then report confidence intervals for the others.

Practical approach: start with ~ 20 replications, inspect the across-replication mean and half-width, increase the count and see whether results move much — if they barely move, you have enough. In practice 30, 50 or 100 replications are usually sufficient; if the half-width is too wide, just add replications.

Key relationship: the *smaller* the half-width you desire, the *more* replications you need.

15. Confidence interval for a KPI

Once n is fixed and the production run is done, you report a confidence interval for each KPI. For $n \geq 30$ (using the standard normal, where $z = \Phi^{-1}(1 - \frac{\alpha}{2}) = 1.96$ for 95% confidence):

$$\bar{x} \pm \frac{z s}{\sqrt{n}}$$

or, equivalently for smaller n , Rossetti's t-based form $\bar{x} \pm t_{\alpha/2, n-1} \frac{s}{\sqrt{n}}$. The half-width is the $\pm$ term. The sample standard deviation is:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Interpretation. The interval is the range in which the true KPI value lies with probability $(1 - \alpha)$. *Example:* the airport production run of 67 replications reported turnaround = 72.61 min with half-width 1.32 $\Rightarrow$ [71.30, 73.93] min — “with 95% confidence the true mean turnaround lies in this range.”

16. Time horizon: terminating vs. steady-state

A key distinction in output analysis is the time horizon of the simulation, because it changes how you analyse the output.

Terminating (finite-horizon). The simulation has *natural, identical starting and ending conditions* for every replication — e.g. a bank that opens empty at 09:00 and closes at 17:00. Analysed by the **method of independent replications**: all replications share the same start/end conditions, and the data are independent *across* replications even if the data *within* a single replication may or may not be independent.

Infinite-horizon (steady-state). The simulation has *no natural endpoint* and is often run as a *single long replication*. Here the within-replication data are typically *not* i.i.d. (they are autocorrelated), and the central concern becomes the **warm-up period** — the initial transient that must be *discarded* before the system reaches steady state and representative data can be collected. The question “how long must it run before steady-state data can be collected?” is the defining concern of this case.

17. Comparing two system configurations

The point of the whole study is usually to *choose* between designs. To compare two configurations on a KPI, run n replications of each — ideally with a **common random number** stream so the comparison is *paired* and fair. For each replication j compute the difference:

$$D_j = X_{1j} - X_{2j}, \quad \bar{D} = \frac{1}{n} \sum_{j=1}^n D_j$$

Then the sample standard deviation of the differences and the $(1 - \alpha)$ confidence interval on the difference:

$$S_D = \sqrt{\frac{1}{n-1} \sum_{j=1}^n (D_j - \bar{D})^2}, \quad \bar{D} \pm t_{\alpha/2, n-1} \frac{S_D}{\sqrt{n}}$$

Three possible outcomes:

1. Entire interval *above* 0 $\Rightarrow$ configuration 1 is statistically better on that KPI.
2. Entire interval *below* 0 $\Rightarrow$ configuration 2 is statistically better.
3. Interval *contains* 0 $\Rightarrow$ no statistical decision can be made; you need more replications.

Practical coda. If the statistical difference is small but one configuration is cheaper or simpler to implement, choose that one. The statistical comparison is a *tool to strengthen your argument*, not a replacement for sound business judgement.

18. Experimental design (Phase 3)

Experimental design governs how you explore the space of input factors. From an operations-management view its purpose is to learn how the system responds to changes in controllable factors, so you can recommend a design. You typically organise the comparison around a **base case** scenario plus a set of alternative scenarios measured against it. There are three approaches.

- **OFAT (One Factor At a Time).** Vary one factor while holding all others fixed, then move to the next factor. *Pros:* simple to understand. *Cons:* time-consuming and ineffective when there are many factors (many runs needed); and critically it *misses interactions* between factors, because it never varies two together — so it can mislead you about how the response really behaves.
- **2^K factorial design.** Considers $K \geq 2$ factors *simultaneously*, setting *two levels* (a low and a high) for each factor to define the extreme design points. *Advantages:* it *detects interactions* between factors (fewer misunderstandings than OFAT), and it lets a factor's effect be estimated across several levels of the other factors. *Assumption:* the response is approximately linear (monotonic) over each factor's range, so the two chosen levels must be realistic yet far enough apart to reveal a difference.
- **Simulation-based optimisation.** Define an *objective function* over selected output performance measures (e.g. profit, cost, waiting time) subject to *constraints* (e.g. average waiting time below 5 minutes, or utilisation at least 80%), and search for the input combination that optimises it. Used when there are many factors and a clear performance objective.

19. Tutorial & exercise recap (testable)

The exam slide lists *the tutorials/exercises* as a separate testable source. Below is what each tutorial taught — the problem framing, the modelling choices, and the conclusions — the kind of thing a multiple-choice question would draw on.

Bank Office tutorial (AnyLogic “Getting Started”)

Customers arrive at a bank and either use a single **ATM** or are served by one of four **tellers**; a single FIFO queue feeds each. The tutorial's hidden lesson is that AnyLogic shows you *how* to build the model but not *why*, so you reverse-engineer Phase 1: the management's real goal is to balance **service level against operational cost** — offer low waiting times while not over-paying for resources. The two candidate **bottlenecks** are the number of tellers and the single ATM;

cutting tellers from four to three would raise waiting time (worse service), while rising ATM demand might justify a second ATM. The suggested **KPIs** (built with SMART) were: ATM utilisation (%), average queue length (customers), waiting-time distribution (per customer), and time in system (per customer); teller utilisation is also worth tracking under varying demand. The entity is the customer (the modeller cares about the customer's "life" in the system); the resources are the tellers and ATM. There is *no single correct solution* — the framing matters more than exact numbers.

YummiBar production line

A chocolate-bar line: dough → mixer → moulder → cooler → packaging (wrap, box of 24, stack of 20 boxes). The firm owns 1 mixer, 2 moulders, 3 coolers, and two packaging lines (one old, one new at double capacity). Capacities (bars/h): mixer 4000, moulder 1200, cooler 1100, old packaging 1000, new packaging 2000. Dough arrives at 150 kg/h and bars weigh 50 g, so ≈ 3000 bars/h enter; the factory runs 12 h/day.

Key modelling decisions (likely MCQ material):

- **Level of detail / agent unit.** Modelling each individual bar is too fine. The chosen entity is a **tray of 20 bars** (the box size); a second agent, a **pallet** of 24 trays = 480 bars, models the final stage. 480 was rejected as the base unit because several machine capacities aren't divisible by 480 and a unit must be processed whole.
- **Delay time** per machine = $\frac{1}{\text{capacity}/20}$ (because one unit = 20 bars). Each machine processes one unit at a time.
- **Time unit** = minutes (seconds too granular for a 12-h run; hours too coarse).
- **Build incrementally** — one machine at a time, testing as you go; you can't get TA help without an activity diagram first.

Conclusion: throughput of a serial line equals the **slowest stage**. The new branch is limited by the moulder (1200), the old branch by old packaging (1000). Since the CFO allows replacing *one* machine, the right move is to relieve the binding stage on the busiest path — upgrading **Moulder 1** gives the best single-machine return, because upgrading a non-bottleneck machine does nothing for output.

Airport ground handling case

Aircraft (the entities) land, are allocated to **gates** (resources), and undergo ground handling before departing. The system starts when an aircraft enters the airport / transfer queue and ends when it leaves its gate. Management believes the **gates are the bottleneck**, so the study tracks queue times and utilisation to confirm and relieve it. The case is built up over three days: (1) gate allocation + refuelling, (2) baggage handling (after refuelling, needs a baggage truck) and a varying number of fuel trucks, (3) input modelling + output analysis.

Key modelling details:

- **Refuelling delay** depends on tank size: passengers $\geq 150 \Rightarrow$ tank 200,000, else 150,000; delay = (tank capacity – residual fuel) / refuelling speed. The aircraft *seizes* a fuel truck (a *dynamic* resource pool, e.g. 2 trucks) while the gate is a *static* pool.
- **Input modelling** of the two stochastic processes: fit a distribution to `RefuellingData.xlsx` (time per litre) and `BaggageHandlingData.xlsx` (time per bag) using the Data Distribution Fitter.
- **Downtime block** models a stuck bag: inter-arrival 30 min, duration $\sim N(\mu = 90\text{s}, \sigma = \mu/10)$ — a ramp agent manually intervenes.

- **Two datasets:** `TestSchedule.xlsx` (small) for building/testing; `FullSchedule.xlsx` (more flights, peak-hour traffic) for the real runs.
- **Verification checks used:** #agents in = #agents out; agents loaded correctly from the arrival scheme; operations in the right order; resources seized/released properly; observed delay matches the distribution mean — aided by console prints and stress-testing.
- **An assumption that matters:** wait time *between* two operations was counted as operation time, not queue time — so if all fuel trucks are busy the queue time is understated. Changing this requires updating both the activity diagram and the assumptions. (Lesson: results depend on how you defined your KPIs and assumptions.)

Results (95% CI, 67 replications): turnaround time 72.61 ± 1.32 min, queue time 21.03 ± 0.64 min, gate utilisation $58.4\% \pm 0.7\%$ — consistent, low-variability operation; the airport is in steady state.

Cross-cutting lessons

All three followed **DEGREE + the six-phase methodology**, did Phase 1 (problem, system, KPIs, conceptual model, assumptions) *before* coding, and used the same building blocks. Recurring exam-relevant points: the entity is whatever “flows” and whose journey you care about; resources are what constrain that flow; the bottleneck is the slowest stage and the only worthwhile upgrade target; KPI definitions and assumptions directly shape the results; and there is rarely one “correct” model — the abstraction level must fit the question.

Final exam reminders

The exam tests the *conceptual* curriculum (Rossetti sections per the course plan, the lectures, the tutorials/exercises, input modelling and output analysis). It is *not* a test in AnyLogic, but questions may assume the AnyLogic understanding from the Bank Office, YummiBar and Airport exercises. Pen-and-paper multiple choice, no aids except your one note sheet (both sides allowed). Highest-probability targets: *verification vs. validation*, the *six phases*, the *goodness-of-fit “can reject but not rank”* point, the *three comparison outcomes*, and not confusing the *experienced* half-width h_0 with the *desired* half-width h . Good luck.